

LAPORAN PRATIKUM STRUKTUR DATA

LAPORAN PRAKTIKUM PEKAN 6

Disusun Oleh:

Nama: Muhammad Aufa Rafiki

NIM: 2511531012

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum ini dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW yang telah membawa umat manusia ke zaman yang penuh ilmu pengetahuan.

Laporan praktikum ini disusun untuk membahas salah satu materi dalam Struktur Data, yaitu Doubly Linked List. Pada laporan ini dijelaskan konsep dasar Doubly Linked List, struktur node yang terdiri dari data serta pointer next dan prev, serta implementasi berbagai operasi dasar seperti penambahan node di awal, akhir, dan posisi tertentu, penghapusan node, serta proses penelusuran data baik secara maju maupun mundur.

Melalui praktikum ini, penulis mempelajari bagaimana cara mengelola data secara dinamis menggunakan struktur data yang saling terhubung, serta memahami bagaimana hubungan antar node bekerja dalam sebuah Doubly Linked List. Selain itu, laporan ini juga memuat contoh implementasi menggunakan bahasa pemrograman Java untuk memperjelas pemahaman terhadap materi yang dibahas.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan untuk perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat dan menambah pemahaman pembaca mengenai Doubly Linked List.

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 NodeDLL_2511531012.....	2
2.3 InsertDLL_2511531012	3
2.4 PenelusuranDLL_2511531012.....	5
2.5 HapusDLL_2511531012	7
BAB III KESIMPULAN	9
DAFTAR PUSTAKA	10
Tugas	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara efisien. Salah satu jenis struktur data yang sering digunakan adalah linked list, khususnya Doubly Linked List.

Doubly Linked List memiliki keunggulan karena setiap node dapat diakses secara dua arah, yaitu ke node berikutnya dan node sebelumnya. Hal ini memudahkan dalam proses penelusuran, penyisipan, dan penghapusan data. Oleh karena itu, pemahaman mengenai Doubly Linked List sangat penting bagi mahasiswa dalam mempelajari struktur data.

1.2 Tujuan Pratikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar Doubly Linked List
2. Mampu mengimplementasikan struktur Doubly Linked List dalam bahasa Java
3. Mengetahui cara menambahkan node (di awal, akhir, dan posisi tertentu)
4. Mengetahui cara menghapus node (head, akhir, dan posisi tertentu)
5. Mampu melakukan penelusuran data (maju dan mundur) dalam Doubly Linked List

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Menambah pemahaman tentang konsep Doubly Linked List
2. Melatih kemampuan logika dan pemecahan masalah dalam pemrograman
3. Membantu memahami pengelolaan data secara dinamis dan terstruktur
4. Meningkatkan keterampilan dalam mengimplementasikan struktur data menggunakan bahasa Java
5. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks di masa mendatang

BAB II

PEMBAHASAN

2.1 Dasar teori

Double Linked List adalah salah satu bentuk struktur data linked list di mana setiap node memiliki dua pointer, yaitu pointer next yang menunjuk ke node berikutnya dan pointer prev yang menunjuk ke node sebelumnya. Struktur ini memungkinkan proses penelusuran data dilakukan dalam dua arah, yaitu dari depan ke belakang (forward traversal) maupun dari belakang ke depan (backward traversal). Double linked list banyak digunakan karena memudahkan operasi penyisipan, penghapusan, dan penelusuran node pada posisi tertentu tanpa harus selalu memulai dari awal list. Namun, struktur ini memiliki kelemahan yaitu penggunaan memori yang lebih besar dibandingkan single linked list, karena setiap node harus menyimpan dua referensi (pointer) sekaligus.

2.2 NodeDLL_2511531012

```
1 package pekan6_2511531012;
2
3 public class NodeDLL_2511531012 {
4     //mendefenisikan kelas node
5     int data_1012;
6     NodeDLL_2511531012 next_1012; // pointer ke next node
7     NodeDLL_2511531012 prev_1012; // pointer ke previous node
8
9     // konstruktor
10    public NodeDLL_2511531012(int data_1012) {
11        this.data_1012 = data_1012;
12        this.next_1012 = null;
13        this.prev_1012 = null;
14    }
15 }
16
```

Penjelasan Program:

1. public class NodeDLL_2511531012 → Membuat kelas node untuk Doubly Linked List.
2. int data_1012; → Menyimpan nilai data pada node.
3. NodeDLL_2511531012 next_1012; → Pointer ke node berikutnya.
4. NodeDLL_2511531012 prev_1012; → Pointer ke node sebelumnya.
5. public NodeDLL_2511531012(int data_1012) → Constructor untuk membuat node baru dengan nilai awal.
6. this.data_1012 = data_1012; → Mengisi data node dengan nilai yang dikirim.
7. this.next_1012 = null; → Inisialisasi next belum terhubung.
8. this.prev_1012 = null; → Inisialisasi prev belum terhubung.

2.3 InsertDLL_2511531012

```
1 package pekan6_2511531012;
2
3 public class InsertDLL_2511531012 {
4     //menambahkan node di awal dll
5     static NodeDLL_2511531012 insertBegin_1012(NodeDLL_2511531012 head_1012,int data_1012) {
6         //buat node baru
7         NodeDLL_2511531012 new_node_1012 = new NodeDLL_2511531012(data_1012);
8         //jadikan pointer nextnya head
9         new_node_1012.next_1012 = head_1012;
10        //jadikan pointer prev head ke new_node
11        if(head_1012 != null) {
12            head_1012.prev_1012 = new_node_1012;
13        }
14        return new_node_1012;
15    }
16    //fungsi menambahkan node di akhir
17    public static NodeDLL_2511531012 insertEnd_1012(NodeDLL_2511531012 head_1012, int newData_1012) {
18        //buat node baru
19        NodeDLL_2511531012 newNode_1012 = new NodeDLL_2511531012(newData_1012);
20        //jika dll null jadikan head
21        if(head_1012 == null) {
22            head_1012 = newNode_1012;
23        }
24        else {
25            NodeDLL_2511531012 curr_1012 = head_1012;
26            while (curr_1012.next_1012 != null) {
27                curr_1012 = curr_1012.next_1012;
28            }
29            curr_1012.next_1012 = newNode_1012;
30            newNode_1012.prev_1012 = curr_1012;
31        }
32        return head_1012;
33    }
34    //fungsi menambahkan node di posisi tertentu
35    public static NodeDLL_2511531012 insertAtPosition(NodeDLL_2511531012 head_1012, int pos_1012, int new_data_1012) {
36        // buat node baru
37        NodeDLL_2511531012 new_node_1012 = new NodeDLL_2511531012(new_data_1012);
38        if(pos_1012 == 1) {
39            new_node_1012.next_1012 = head_1012;
40            if (head_1012 != null) {
41                head_1012.prev_1012 = new_node_1012;
42            }
43            return head_1012;
44        }
45        NodeDLL_2511531012 curr_1012 = head_1012;
46        for (int i = 1; i < pos_1012 - 1 && curr_1012 != null;i++ ) {
47            curr_1012 = curr_1012.next_1012;
48        }
49        if (curr_1012 == null) {
50            System.out.println("Posisi tidak ada");
51            return head_1012;
52        }
53        new_node_1012.prev_1012 = curr_1012;
54        new_node_1012.next_1012 = curr_1012.next_1012;
55        curr_1012.next_1012 = new_node_1012;
56        if(new_node_1012.next_1012 != null) {
57            new_node_1012.next_1012.prev_1012 = new_node_1012;
58        }
59        return head_1012;
60    }
61    public static void printList_1012(NodeDLL_2511531012 head_1012) {
62        NodeDLL_2511531012 curr_1012 = head_1012;
63        while (curr_1012 != null) {
64            System.out.print(curr_1012.data_1012 + " <-> ");
65            curr_1012 = curr_1012.next_1012;
66        }
67        System.out.println();
68    }
69    public static void main(String[]args) {
70        //membuat dll 2<->3<->5<->
71        NodeDLL_2511531012 head_1012 = new NodeDLL_2511531012(2);
72        head_1012.next_1012 = new NodeDLL_2511531012(3);
73        head_1012.next_1012.prev_1012 = head_1012;
74        head_1012.next_1012.next_1012 = new NodeDLL_2511531012(5);
75        head_1012.next_1012.prev_1012.prev_1012 = head_1012;
76
77        System.out.print("DLL Awal: ");
78        printList_1012(head_1012);
79
80        head_1012 = insertBegin_1012(head_1012,1);
81        System.out.print("simpul 1 ditambahkan di awal: ");
82        printList_1012(head_1012);
83
84        System.out.print("simpul 6 ditambahkan di akhir: ");
85        int data_1012=6;
86        head_1012 = insertEnd_1012(head_1012, data_1012);
87        printList_1012(head_1012);
88
89        System.out.print("tambahkan node 4 di posisi 4: ");
90        int data2_1012 = 4;
91        int pos_1012 = 4;
92        head_1012 = insertAtPosition(head_1012, pos_1012, data2_1012);
93        printList_1012(head_1012);
94    }
95 }
```

Penjelasan Program:

1. Method insertBegin_1012(..)
 - new NodeDLL_2511531012(data_1012); → Membuat node baru
 - new_node_1012.next_1012 = head_1012; → Menghubungkan node baru ke head lama
 - if(head_1012 != null) → Mengecek apakah list tidak kosong
 - head_1012.prev_1012 = new_node_1012; → Menghubungkan head lama ke node baru
 - return new_node_1012; → Mengembalikan node baru sebagai head
2. Method insertEnd_1012(...)
 - new NodeDLL_2511531012(newData_1012); → Membuat node baru
 - if(head_1012 == null) → Jika list kosong
 - head_1012 = newNode_1012; → Node baru jadi head
 - while(curr_1012.next_1012 != null) → Mencari node terakhir
 - curr_1012.next_1012 = newNode_1012; → Menghubungkan node terakhir ke node baru
 - newNode_1012.prev_1012 = curr_1012; → Menghubungkan node baru ke node sebelumnya
3. Method insertAtPosition(...)
 - new NodeDLL_2511531012(new_data_1012); → Membuat node baru
 - if(pos_1012 == 1) → Jika posisi di awal
 - new_node_1012.next_1012 = head_1012; → Hubungkan ke head lama
 - head_1012.prev_1012 = new_node_1012; → Hubungkan balik ke node baru
 - for(...) → Mencari posisi sebelum insert
 - if(curr_1012 == null) → Jika posisi tidak valid
 - new_node_1012.prev_1012 = curr_1012; → Hubungkan ke node sebelumnya
 - new_node_1012.next_1012 = curr_1012.next_1012; → Hubungkan ke node setelahnya
 - curr_1012.next_1012 = new_node_1012; → Menyisipkan node baru
 - new_node_1012.next_1012.prev_1012 = new_node_1012; → Menghubungkan node setelahnya supaya menunjuk kembali ke node baru yang ditambahkan
4. Method printList_1012(...)
 - curr_1012 = head_1012; → Mulai dari head
 - while(curr_1012 != null) → Traversing sampai akhir list
 - System.out.print(data + " <-> "); → Menampilkan data tiap node
 - curr_1012 = curr_1012.next_1012; → Pindah ke node berikutnya
 - System.out.println(); → Pindah baris setelah selesai
5. Method main(..)
 - Membuat linked list awal 2 <-> 3 <-> 5
 - insertBegin_1012(head_1012, 1) → Menambah node (1) di awal
 - insertEnd_1012(head_1012, 6) → Menambah node (6) di akhir
 - insertAtPosition(head_1012, 4, 4) → Menambah node (4) di posisi ke-4
 - printList_1012(head_1012) → Menampilkan hasil setiap perubahan

Output:

```
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambahkan di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambahkan di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambahkan node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.4 PenelusuranDLL_2511531012

```
1 package pekan6_2511531012;
2
3 public class PenelusuranDLL_2511531012 {
4     //fungsi penelusuran maju
5     static void forwardTraversal_1012(NodeDLL_2511531012 head_1012) {
6         // memulai penelusuran dari head
7         NodeDLL_2511531012 curr_1012 = head_1012;
8         //lanjutkan sampai akhir
9         while (curr_1012 != null) {
10             //print data
11             System.out.print(curr_1012.data_1012 + " <-> ");
12             //pindah ke node berikutnya
13             curr_1012 = curr_1012.next_1012;
14         }
15         //print spasi
16         System.out.println();
17     }
18     //fungsi penelusuran mundur
19     static void backwardTraversal_1012(NodeDLL_2511531012 tail_1012) {
20         //mulai dari akhir
21         NodeDLL_2511531012 curr_1012 = tail_1012;
22         //lanjutkan sampai head
23         while (curr_1012 != null) {
24             //cetak data
25             System.out.print(curr_1012.data_1012 + " <-> ");
26             //pindah ke node berikutnya
27             curr_1012 = curr_1012.prev_1012;
28         }
29         //cetak spasi
30         System.out.println();
31     }
32
33     public static void main(String[] args) {
34         //cetak dll
35         NodeDLL_2511531012 head_1012 = new NodeDLL_2511531012(1);
36         NodeDLL_2511531012 second_1012 = new NodeDLL_2511531012(2);
37         NodeDLL_2511531012 third_1012 = new NodeDLL_2511531012(3);
38
39         head_1012.next_1012 = second_1012;
40         second_1012.prev_1012 = head_1012;
41         second_1012.next_1012 = third_1012;
42         third_1012.prev_1012 = second_1012;
43
44         System.out.println("Penelusuran maju: ");
45         forwardTraversal_1012(head_1012);
46
47         System.out.println("Penelusuran mundur: ");
48         backwardTraversal_1012(third_1012);
49     }
50
51 }
52
```

Penjelasan Program:

1. Method forwardTraversal_1012(...)
 - curr_1012 = head_1012; → Mulai dari node pertama (head)
 - while(curr_1012 != null) → Jalan sampai akhir list
 - System.out.print(...) → Menampilkan data node
 - curr_1012 = curr_1012.next_1012; → Pindah ke node berikutnya
 - System.out.println(); → Pindah baris setelah selesai
2. Method backwardTraversal_1012(...)
 - curr_1012 = tail_1012; → Mulai dari node terakhir (tail)
 - while(curr_1012 != null) → Jalan sampai awal list
 - System.out.print(...) → Menampilkan data node
 - curr_1012 = curr_1012.prev_1012; → Pindah ke node sebelumnya
 - System.out.println(); → Pindah baris setelah selesai
3. Method main(...)
 - new NodeDLL_2511531012(1/2/3) → Membuat 3 node
 - head.next = second → Menghubungkan node 1 ke node 2
 - second.prev = head → Menghubungkan balik node 2 ke node 1
 - second.next = third → Menghubungkan node 2 ke node 3
 - third.prev = second → Menghubungkan balik node 3 ke node 2
 - forwardTraversal(head) → Menampilkan dari depan ke belakang (1 → 2 → 3)
 - backwardTraversal(third) --> Menampilkan dari belakang ke depan (3 → 2 → 1)

Output:

```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

2.5 HapusDLL_2511531012

```
1 package pekan6_2511531012;
2
3 public class HapusDLL_2511531012 {
4
5     public static NodeDLL_2511531012 delHead_1012(NodeDLL_2511531012 head_1012) {
6         if(head_1012 == null) {
7             return null; }
8         head_1012 = head_1012.next_1012;
9         if(head_1012 != null) {
10             head_1012.prev_1012 = null; }
11         return head_1012;
12     }
13
14     public static NodeDLL_2511531012 delLast_1012(NodeDLL_2511531012 head_1012) {
15         if(head_1012 == null ) {
16             return null; }
17         if(head_1012.next_1012 == null) {
18             return null; }
19         NodeDLL_2511531012 curr_1012 = head_1012;
20         while (curr_1012.next_1012 != null) {
21             curr_1012 = curr_1012.next_1012;
22         }
23
24         if(curr_1012.prev_1012 != null) {
25             curr_1012.prev_1012.next_1012 = null; }
26         return head_1012;
27     }
28
29     public static NodeDLL_2511531012 delPos_1012(NodeDLL_2511531012 head_1012, int pos_1012) {
30         if (head_1012 == null) {
31             return head_1012; }
32         NodeDLL_2511531012 curr_1012 = head_1012;
33         for (int i = 1; curr_1012 != null && i < pos_1012; i++) {
34             curr_1012 = curr_1012.next_1012;
35         }
36         if (curr_1012 == null) {
37             return head_1012; }
38
39         if (curr_1012.prev_1012 != null) {
40             curr_1012.prev_1012.next_1012 = curr_1012.next_1012; }
41         if (curr_1012.next_1012 != null) {
42             curr_1012.next_1012.prev_1012 = curr_1012.prev_1012; }
43
44         if (head_1012 == curr_1012) {
45             head_1012 = curr_1012.next_1012;
46         }
47         return head_1012;
48     }
49
50     public static void printlist_1012(NodeDLL_2511531012 head_1012) {
51         NodeDLL_2511531012 curr_1012 = head_1012;
52         while (curr_1012 != null) {
53             System.out.print(curr_1012.data_1012 + " <-> ");
54             curr_1012 = curr_1012.next_1012;
55         }
56         System.out.println();
57     }
58
59     public static void main(String[] args) {
60
61         NodeDLL_2511531012 head_1012 = new NodeDLL_2511531012(1);
62         head_1012.next_1012 = new NodeDLL_2511531012(2);
63         head_1012.next_1012.prev_1012 = head_1012;
64         head_1012.next_1012.next_1012 = new NodeDLL_2511531012(3);
65         head_1012.next_1012.next_1012.prev_1012 = head_1012.next_1012;
66         head_1012.next_1012.next_1012.next_1012 = new NodeDLL_2511531012(4);
67         head_1012.next_1012.next_1012.next_1012.prev_1012 = head_1012.next_1012.next_1012;
68         head_1012.next_1012.next_1012.next_1012.next_1012 = new NodeDLL_2511531012(5);
69         head_1012.next_1012.next_1012.next_1012.next_1012.prev_1012 = head_1012.next_1012.next_1012.next_1012;
```

```

69     System.out.print("DLL Awal: ");
70     printList_1012(head_1012);
71
72     System.out.print("Setelah head dihapus: ");
73     head_1012 = delHead_1012(head_1012);
74     printList_1012(head_1012);
75
76     System.out.print("Setelah node terakhir dihapus: ");
77     head_1012 = delLast_1012(head_1012);
78     printList_1012(head_1012);
79
80     System.out.print("menghapus node ke 2: ");
81     head_1012 = delPos_1012(head_1012,2);
82     printList_1012(head_1012);
83 }
84
85 }
86

```

Penjelasan Program:

1. Method delHead_1012(...)
 - if(head_1012 == null) → Jika list kosong, langsung berhenti
 - head_1012 = head_1012.next_1012; → Geser head ke node berikutnya
 - if(head_1012 != null) → Jika masih ada node
 - head_1012.prev_1012 = null; → Putuskan hubungan ke node lama
 - return head_1012; → Kembalikan head baru
2. Method delLast_1012(...)
 - if(head_1012 == null) → Jika list kosong
 - if(head_1012.next_1012 == null) → Jika hanya 1 node
 - return null; → List jadi kosong
 - while(curr_1012.next_1012 != null) → Cari node terakhir
 - curr_1012.prev_1012.next_1012 = null; → Hapus hubungan ke node terakhir
 - return head_1012; → Head tetap sama
3. Method delPos_1012(...)
 - if(head_1012 == null) → Jika list kosong
 - for(...) → Mencari node di posisi yang mau dihapus
 - if(curr_1012 == null) → Jika posisi tidak ada
 - curr_1012.prev_1012.next_1012 = curr_1012.next_1012; → Hubungkan node sebelum ke node sesudah
 - curr_1012.next_1012.prev_1012 = curr_1012.prev_1012; → Hubungkan balik node sesudah ke node sebelum
 - if(head_1012 == curr_1012) → Jika yang dihapus adalah head
 - head_1012 = curr_1012.next_1012; → Geser head ke depan
4. Method main(...)
 - Membuat DLL awal 1 <-> 2 <-> 3 <-> 4 <-> 5
 - delHead_1012(head_1012) → Menghapus node pertama (1)
 - delLast_1012(head_1012) → Menghapus node terakhir (5)
 - delPos_1012(head_1012, 2) → Menghapus node di posisi ke-2
 - printList_1012(head_1012) → Menampilkan hasil setiap perubahan

Output:

```

DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
menghapus node ke 2: 2 <-> 4 <->

```

BAB III

KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung melalui pointer atau referensi. Setiap node memiliki dua komponen utama, yaitu data dan pointer yang menunjuk ke node berikutnya, sehingga memungkinkan data disusun secara fleksibel tanpa harus berada pada lokasi memori yang berurutan. Melalui implementasi program menggunakan bahasa Java, operasi dasar seperti penambahan node di awal, di akhir, maupun pada posisi tertentu dapat dilakukan dengan baik sesuai dengan konsep yang telah dipelajari.

Selain itu, operasi penghapusan node, pencarian data, serta penelusuran (traversal) juga dapat dijalankan dengan cara mengatur hubungan antar node melalui pointer. Hal ini menunjukkan bahwa Linked List memiliki keunggulan dalam hal fleksibilitas dan efisiensi dalam manipulasi data dibandingkan struktur data statis. Dengan demikian, pemahaman mengenai Single Linked List sangat penting sebagai dasar dalam mempelajari struktur data yang lebih kompleks serta dalam pengembangan program yang membutuhkan pengelolaan data secara dinamis dan terstruktur.

DAFTAR PUSTAKA

- [1] I. Liem, *Draft Diktat Struktur Data*, Edisi 2008. Bandung, Indonesia: Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.

Tugas

Lagu_2511531012

```
1 package pekan6_2511531012;
2
3 public class Lagu_2511531012 {
4     String judul_1012;
5     String penyanyi_1012;
6     Lagu_2511531012 next_1012;
7     Lagu_2511531012 prev_1012;
8
9     // constructor
10 public Lagu_2511531012(String judul_1012, String penyanyi_1012) {
11     this.judul_1012 = judul_1012;
12     this.penyanyi_1012 = penyanyi_1012;
13     this.next_1012 = null;
14     this.prev_1012 = null;
15 }
16
17 // GETTER
18 public String getJudul_1012() { return judul_1012; }
19 public String getPenyanyi_1012() { return penyanyi_1012; }
20 public Lagu_2511531012 getNext_1012() { return next_1012; }
21 public Lagu_2511531012 getPrev_1012() { return prev_1012; }
22
23 // SETTER
24 public void setJudul_1012(String judul_1012) { this.judul_1012 = judul_1012; }
25 public void setPenyanyi_1012(String penyanyi_1012) { this.penyanyi_1012 = penyanyi_1012; }
26 public void setNext_1012(Lagu_2511531012 next_1012) { this.next_1012 = next_1012; }
27 public void setPrev_1012(Lagu_2511531012 prev_1012) { this.prev_1012 = prev_1012; }
28 }
29 }
```

Penjelasan Program:

1. Deklarasi Kelas → `public class Lagu_2511531012` → Membuat kelas untuk menyimpan data lagu dalam Doubly Linked List
2. Variabel (Atribut):
 - `String judul_1012;` → Menyimpan judul lagu
 - `String penyanyi_1012;` → Menyimpan nama penyanyi
 - `Lagu_2511531012 next_1012;` → Pointer ke lagu berikutnya
 - `Lagu_2511531012 prev_1012;` → Pointer ke lagu sebelumnya
3. Constructor:
 - `public Lagu_2511531012(...)` → Method untuk membuat objek lagu
 - `this.judul_1012 = judul_1012;` → Mengisi judul lagu
 - `this.penyanyi_1012 = penyanyi_1012;` → Mengisi penyanyi
 - `this.next_1012 = null;` → Node belum terhubung ke depan
 - `this.prev_1012 = null;` → Node belum terhubung ke belakang
4. Getter (Mengambil Data):
 - `getJudul_1012()` → Mengambil nilai judul lagu
 - `getPenyanyi_1012()` → Mengambil nama penyanyi
 - `getNext_1012()` → Mengambil node berikutnya
 - `getPrev_1012()` → Mengambil node sebelumnya

5. Setter (Mengubah Data):

- setJudul_1012(...) → Mengubah judul lagu
- setPenyanyi_1012(...) → Mengubah nama penyanyi
- setNext_1012(...) → Menghubungkan ke node berikutnya
- setPrev_1012(...) → Menghubungkan ke node sebelumnya

Musik_2511531012

```
1 package pekan6_2511531012;
2
3 import java.util.Scanner;
4
5 public class Musik_2511531012 {
6     Lagu_2511531012 head_1012 = null;
7     Lagu_2511531012 tail_1012 = null;
8
9     // tambah lagu di akhir
10    void tambahLagu_1012(String judul_1012, String penyanyi_1012) {
11        Lagu_2511531012 newLagu_1012 = new Lagu_2511531012(judul_1012, penyanyi_1012);
12        if (head_1012 == null) {
13            head_1012 = tail_1012 = newLagu_1012;
14        } else {
15            tail_1012.setNext_1012(newLagu_1012);
16            newLagu_1012.setPrev_1012(tail_1012);
17            tail_1012 = newLagu_1012;
18        }
19        System.out.println("Lagu berhasil ditambahkan!");
20    }
21
22    // hapus lagu awal (head)
23    void hapusLaguAwal_1012() {
24        if (head_1012 == null) {
25            System.out.println("Playlist kosong!");
26            return;
27        }
28        head_1012 = head_1012.getNext_1012();
29
30        if (head_1012 != null) {
31            head_1012.setPrev_1012(null);
32        } else {
33            tail_1012 = null;
34        }
35        System.out.println("Lagu pertama berhasil dihapus!");
36    }
37 }
```

```

36
37 // tampil maju
38 void tampilMaju_1012() {
39     Lagu_2511531012 curr_1012 = head_1012;
40
41     if (curr_1012 == null) {
42         System.out.println("Playlist kosong!");
43         return;
44     }
45
46     while (curr_1012 != null) {
47         System.out.println(curr_1012.getJudul_1012() + " - " + curr_1012.getPenyanyi_1012());
48         curr_1012 = curr_1012.getNext_1012();
49     }
50 }
51
52 // tampil mundur
53 void tampilMundur_1012() {
54     Lagu_2511531012 curr_1012 = tail_1012;
55
56     if (curr_1012 == null) {
57         System.out.println("Playlist kosong!");
58         return;
59     }
60
61     while (curr_1012 != null) {
62         System.out.println(curr_1012.getJudul_1012() + " - " + curr_1012.getPenyanyi_1012());
63         curr_1012 = curr_1012.getPrev_1012();
64     }
65 }
66
67 // cari lagu
68 void cariLagu_1012(String judul_1012) {
69     Lagu_2511531012 curr_1012 = head_1012;
70     boolean ketemu_1012 = false;
71
72     while (curr_1012 != null) {
73         if (curr_1012.getJudul_1012().equalsIgnoreCase(judul_1012)) {
74             System.out.println("Lagu ditemukan: " + curr_1012.getJudul_1012() + " - " + curr_1012.getPenyanyi_1012());
75             ketemu_1012 = true;
76             break;
77         }
78         curr_1012 = curr_1012.getNext_1012();
79     }
80
81     if (!ketemu_1012) {
82         System.out.println("Lagu tidak ditemukan!");
83     }
84 }
85
86 // main program
87 public static void main(String[] args) {
88     Scanner input_1012 = new Scanner(System.in);
89     Musik_2511531012 musik_1012 = new Musik_2511531012();
90
91     int pilihan;
92
93     do {
94         System.out.println("\n=== Playlist Musik NIM: 2511531012 ===");
95         System.out.println("1. Tambah Lagu");
96         System.out.println("2. Hapus Lagu Pertama");
97         System.out.println("3. Lihat Playlist (Maju)");
98         System.out.println("4. Lihat Playlist (Mundur)");
99         System.out.println("5. Cari Lagu");
100        System.out.println("6. Keluar");
101        System.out.print("Pilihan: ");

```

```

102     pilihan = input_1012.nextInt();
103     input_1012.nextLine();
104
105     switch (pilihan) {
106     case 1:
107         System.out.print("Judul: ");
108         String judul = input_1012.nextLine();
109         System.out.print("Penyanyi: ");
110         String penyanyi = input_1012.nextLine();
111         musik_1012.tambahLagu_1012(judul, penyanyi);
112         break;
113     case 2:
114         musik_1012.hapusLaguAwal_1012();
115         break;
116     case 3:
117         musik_1012.tampilMaju_1012();
118         break;
119     case 4:
120         musik_1012.tampilMundur_1012();
121         break;
122     case 5:
123         System.out.print("Masukkan judul lagu: ");
124         String cari = input_1012.nextLine();
125         musik_1012.cariLagu_1012(cari);
126         break;
127     case 6:
128         System.out.println("Keluar...");
129         break;
130     default:
131         System.out.println("Pilihan tidak valid!");
132     }
133 } while (pilihan != 6);
134 input_1012.close();
135 }
136 }

```

Penjelasan Program:

1. Deklarasi Variabel:

- Lagu_2511531012 head_1012 = null → Node pertama (awal playlist)
- Lagu_2511531012 tail_1012 = null → Node terakhir (akhir playlist)

2. Method tambahLagu_1012(...)

- Lagu_2511531012 newLagu_1012 = new Lagu_2511531012(...) → Membuat node lagu baru
- if (head_1012 == null) → Jika playlist kosong
- head_1012 = tail_1012 = newLagu_1012 → Lagu jadi awal dan akhir
- else → Jika sudah ada lagu
- tail_1012.setNext_1012(newLagu_1012) → Hubungkan lagu terakhir ke lagu baru
- newLagu_1012.setPrev_1012(tail_1012) → Hubungkan balik ke lagu sebelumnya
- tail_1012 = newLagu_1012 → Update tail jadi lagu baru
- System.out.println(...) → Tampilkan pesan berhasil

3. Method hapusLaguAwal_1012()

- if (head_1012 == null) → Jika playlist kosong
- System.out.println(...) → Tampilkan pesan kosong
- return → Keluar dari fungsi

- head_1012 = head_1012.getNext_1012() → Geser head ke lagu berikutnya
 - if (head_1012 != null) → Jika masih ada lagu
 - head_1012.setPrev_1012(null) → Putus hubungan ke lagu lama
 - else → Jika jadi kosong
 - tail_1012 = null → Tail juga dikosongkan
 - System.out.println(...) → Tampilkan pesan berhasil
4. Method tampilMaju_1012()
- Lagu_2511531012 curr_1012 = head_1012 → Mulai dari awal
 - if (curr_1012 == null) → Jika kosong
 - System.out.println(...) → Tampilkan pesan kosong
 - return → Keluar
 - while (curr_1012 != null) → Loop sampai akhir
 - System.out.println(...) → Tampilkan judul dan penyanyi
 - curr_1012 = curr_1012.getNext_1012() → Pindah ke lagu berikutnya
5. Method tampilMundur_1012()
- Lagu_2511531012 curr_1012 = tail_1012 → Mulai dari akhir
 - if (curr_1012 == null) → Jika kosong
 - System.out.println(...) → Tampilkan pesan kosong
 - return → Keluar
 - while (curr_1012 != null) → Loop sampai awal
 - System.out.println(...) → Tampilkan judul dan penyanyi
 - curr_1012 = curr_1012.getPrev_1012() → Pindah ke lagu sebelumnya
6. Method cariLagu_1012(...)
- Lagu_2511531012 curr_1012 = head_1012 → Mulai dari awal
 - boolean ketemu_1012 = false → Penanda lagu ditemukan atau tidak
 - while (curr_1012 != null) → Loop sampai akhir
 - if (judul sama) → Cek judul lagu
 - System.out.println(...) → Tampilkan lagu ditemukan
 - ketemu_1012 = true → Tandai sudah ketemu
 - break → Hentikan pencarian
 - curr_1012 = curr_1012.getNext_1012() → Lanjut ke lagu berikutnya
 - if (!ketemu_1012) → Jika tidak ditemukan
 - System.out.println(...) → Tampilkan pesan tidak ditemukan
7. Method main(...)
- Scanner input_1012 = new Scanner(System.in) → Input dari user
 - Musik_2511531012 musik_1012 = new Musik_2511531012() → Membuat objek playlist

- int pilihan → Variabel menu
- Menu Program:
- do → Perulangan menu
- Menampilkan pilihan menu (1–6)
- pilihan = input_1012.nextInt() → Input pilihan user
- switch (pilihan) → Menjalankan menu
- Isi Menu:
- case 1 → Tambah lagu
 - Input judul & penyanyi
 - Panggil tambahLagu_1012()
- case 2 → Hapus lagu pertama
 - Panggil hapusLaguAwal_1012()
- case 3 → Tampilkan maju
 - Panggil tampilMaju_1012()
- case 4 → Tampilkan mundur
 - Panggil tampilMundur_1012()
- case 5 → Cari lagu
 - Input judul
 - Panggil cariLagu_1012()
- case 6 → Keluar program
- default → Jika input salah
- while (pilihan != 6) → Ulang sampai keluar
- input_1012.close() → Menutup input

Output:

```

2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: satu bulan
Penyanyi: Bernadya
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: bersenja gurau
Penyanyi: raim laode
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: monokrom
Penyanyi: tulus
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
satu bulan - Bernadya
bersenja gurau - raim laode
monokrom - tulus

=== Playlist Musik NIM: 2511531012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
monokrom - tulus
bersenja gurau - raim laode
satu bulan - Bernadya

=== Playlist Musik NIM: 2511531012 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus!

```

=== Playlist Musik NIM: 2511531012 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 5

Masukkan judul lagu: monokrom

Lagu ditemukan: monokrom - tulus

=== Playlist Musik NIM: 2511531012 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 5

Masukkan judul lagu: diri

Lagu tidak ditemukan!

=== Playlist Musik NIM: 2511531012 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 6

Keluar...